

```

In [ ]: # Ezinne Akabueze, CMOR 220, Spring 2026, Project 6: Modelling infectious diseases
# Project6_Diseases.ipynb
# This coding program is tasked to model the spread of an infectious disease using the SIR model, w
# Last modified: 3/31/26

import numpy as np # Almost replicates matlab matrix commands
import matplotlib.pyplot as plt # Almost replicates matlab plot commands

# Create simpleSIR function for calculatig all the new values of S, I, and R for each day
def simpleSIR(M, alpha, beta, initialval, Tfinal):
    Sval = np.zeros(Tfinal + 1) # creates a zero vector that holds S values for each day
    Ival = np.zeros(Tfinal + 1) # creates a vector of zeros for the I values for each day
    Rval = np.zeros(Tfinal + 1) # zero vector for the R values; populates for each day
    Sval[0] = initialval[0] # sets the first value of S to the initial value of S
    Ival[0] = initialval[1] # sets the first value of I to the initial value of I
    Rval[0] = initialval[2] # sets the first value of R to the initial value of R
    #loop over the time time steps
    for i in range(Tfinal): # creates a loop that goes through each day from 1 to Tfinal
        Sval[i+1]=Sval[i]-((alpha/M)*Sval[i]*Ival[i])
        Rval[i+1]=Rval[i]+(beta*Ival[i]) #
        Ival[i+1] = Ival[i]+((alpha/M)*Sval[i]*Ival[i]) - (beta*Ival[i])
        # calculates the new S, I, and R values for each day using the equations given in the proje
    return Sval, Rval, Ival # returns the vectors of S, R, and I values for each day

# Set variables to use in the equations
alpha = 1/2 # how quick recovery is
beta = 1/3 # how quick the disease spreads
M = 7.9*10**6 # total population excluding births and deaths
Tfinal = 150 # number of days
Tval = np.arange(0,Tfinal + 1) # Uses matlab code to make a vector from time 0 to Tfinal with a ste
initialval = np.array ([7.9*10**6-10,10,0]) # sets the S, I, and R inital values in a vector

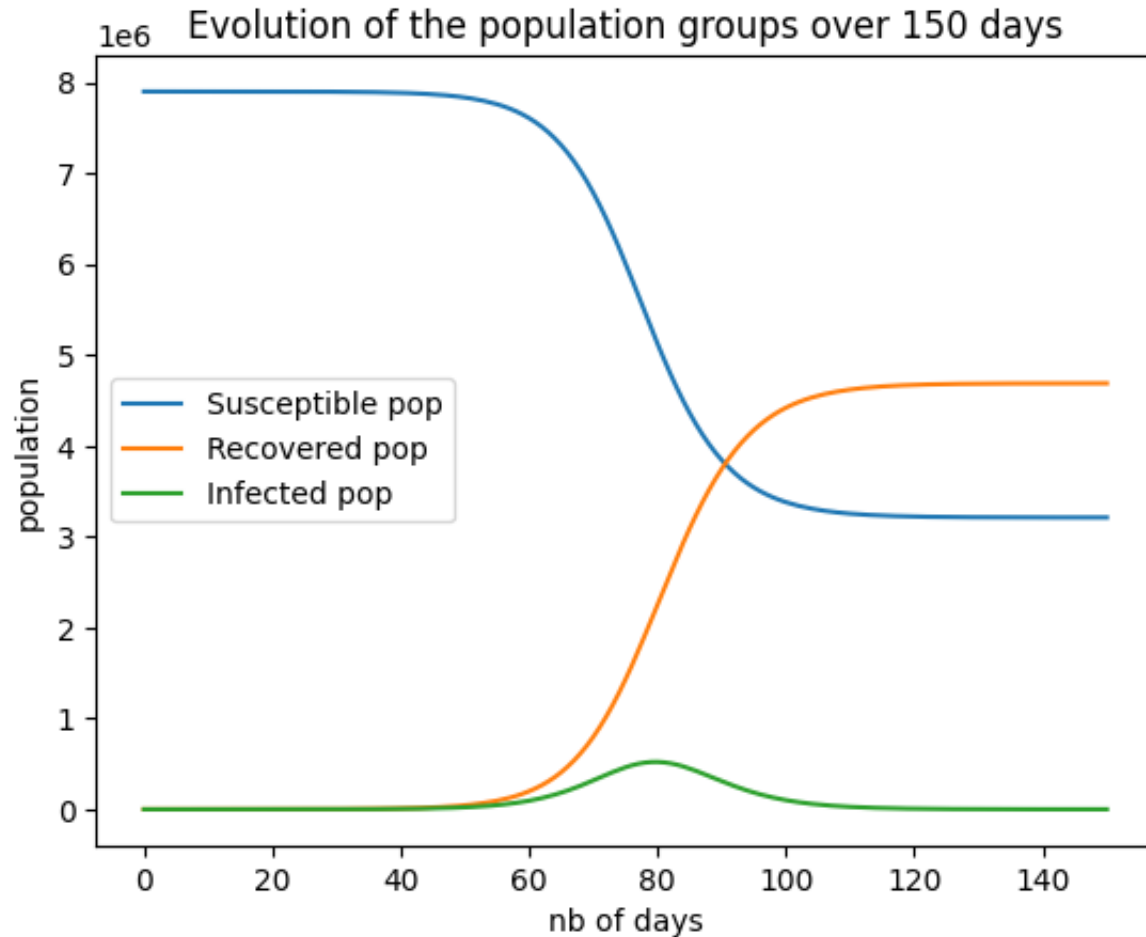
# Run the simpleSIR function to get the S, I, and R values for each day
[Sval,Rval,Ival] = simpleSIR(M,alpha,beta,initialval,Tfinal) # calls created function in order to g

# Plot the S, I, and R values against population vs days
plt.plot(Tval,Sval,label="Susceptible pop")
plt.plot(Tval,Rval,label="Recovered pop")

```

```
plt.plot(Tval,Ival,label="Infected pop")
plt.title('Evolution of the population groups over 150 days')
plt.xlabel('nb of days')
plt.ylabel('population')
plt.legend(loc="center left")
plt.show()

print("S, I, and R change as we change the parameters, such as alpha and beta, because they depend
```



S, I, and R change as we change the parameters, such as alpha and beta, because they depend on one another. When one changes, the others will also change.

```
In [11]: # Define the function for the SIR model that doesn't assume a constant population, so incorporates
```

```

def variableSIR(alpha, beta, mu, initialval, Tfinal, gamma):
    Sval = np.zeros(Tfinal + 1) # creates a zero vector that holds S values for each day
    Ival = np.zeros(Tfinal + 1) # creates a vector of zeros for the I values for each day
    Rval = np.zeros(Tfinal + 1) # zero vector for the R values; populates for each day
    Mval = np.zeros(Tfinal + 1) # zero vector for the total population; populates for each day

    Sval[0] = initialval[0] # sets the first value of S to the initial value of S
    Ival[0] = initialval[1] # sets the first value of I to the initial value of I
    Rval[0] = initialval[2] # sets the first value of R to the initial value of R
    Mval[0] = Sval[0] + Ival[0] + Rval[0] # sets the first value of M to the total population at th

    #loop over the time time steps
    for i in range(Tfinal): # creates a loop that goes through each day from 1 to Tfinal
        Mn = Sval[i] + Ival[i] + Rval[i] # sets M to the total population for each day
        Sval[i+1] = Sval[i] - ((alpha/Mn)*Sval[i]*Ival[i]) + (mu*Mn) - (mu*Sval[i]) # adds the birth rate
        Rval[i+1] = Rval[i] + (beta*Ival[i]) - (mu*Rval[i]) # subtracts the death rate from the R equation
        Ival[i+1] = Ival[i] + ((alpha/Mn)*Sval[i]*Ival[i]) - (beta*Ival[i]) - ((mu + gamma)*Ival[i])

        Mval[i + 1] = Sval[i] + Ival[i] + Rval[i] # updates the total population for each day by adding births and deaths
    return Sval, Rval, Ival, Mval # returns the vectors of S, R, and I values for each day

# Set variables to use in the equations
gamma = 0.02 # recovery rate
mu = 1/(76*365) # death rate (assuming 70-year lifespan)
Tfinal = 4 * 365 # number of days
initialval = np.array ([7.9*10**6-10,10,0]) # sets the S, I, and R initial values in a vector
alpha = 1/2 # how quick recovery is
beta = 1/3 # how quick the disease spreads
Tval = np.arange(0,Tfinal + 1) # Uses matlab code to make a vector from time 0 to Tfinal with a step of 1 day

# Run the variableSIR function to get the S, I, and R values for each day
[Sval,Rval,Ival,Mval] = variableSIR(alpha,beta, mu, initialval,Tfinal, gamma) # calls created function

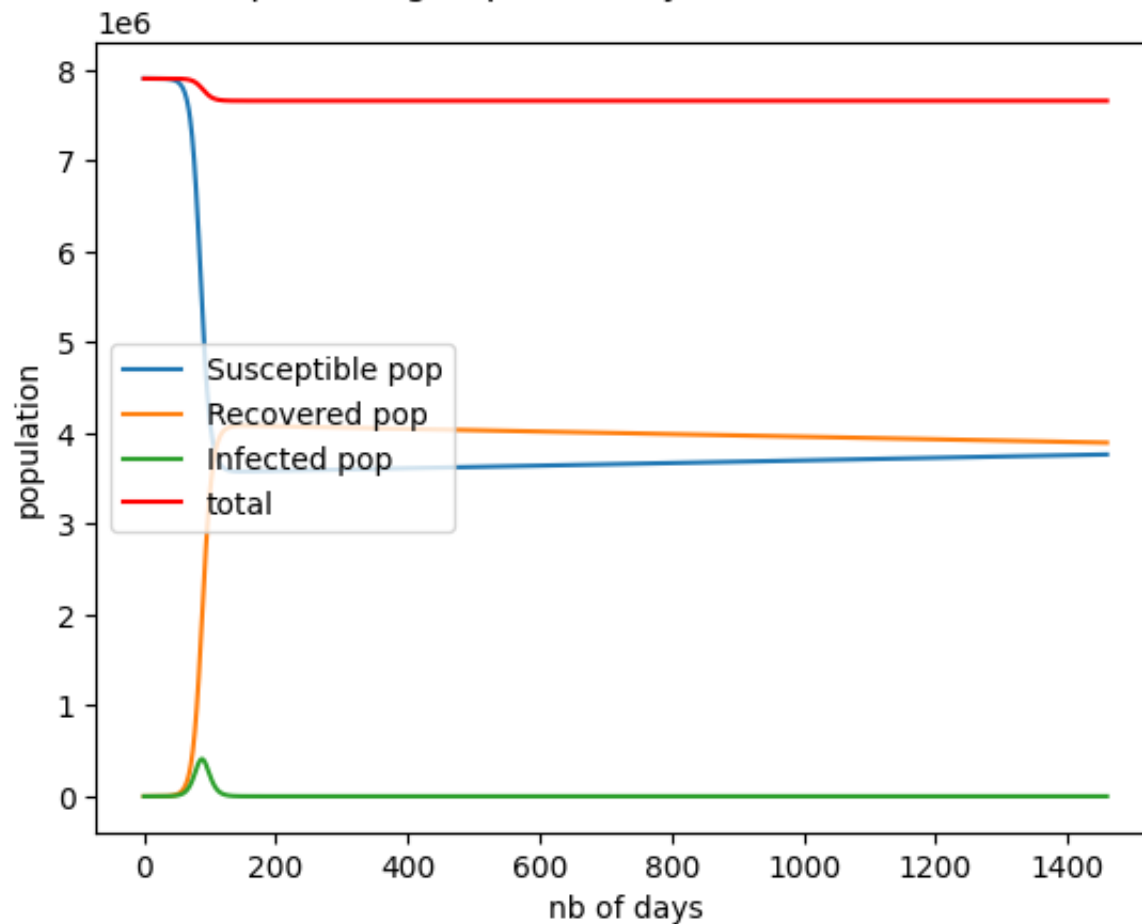
# Plot the S, I, and R values against population vs days
plt.plot(Tval,Sval,label="Susceptible pop")
plt.plot(Tval,Rval,label="Recovered pop")
plt.plot(Tval,Ival,label="Infected pop")
plt.plot(Tval, Mval, label="total", color='red')
plt.title('Part 2: Population groups over 4 years with births and deaths')

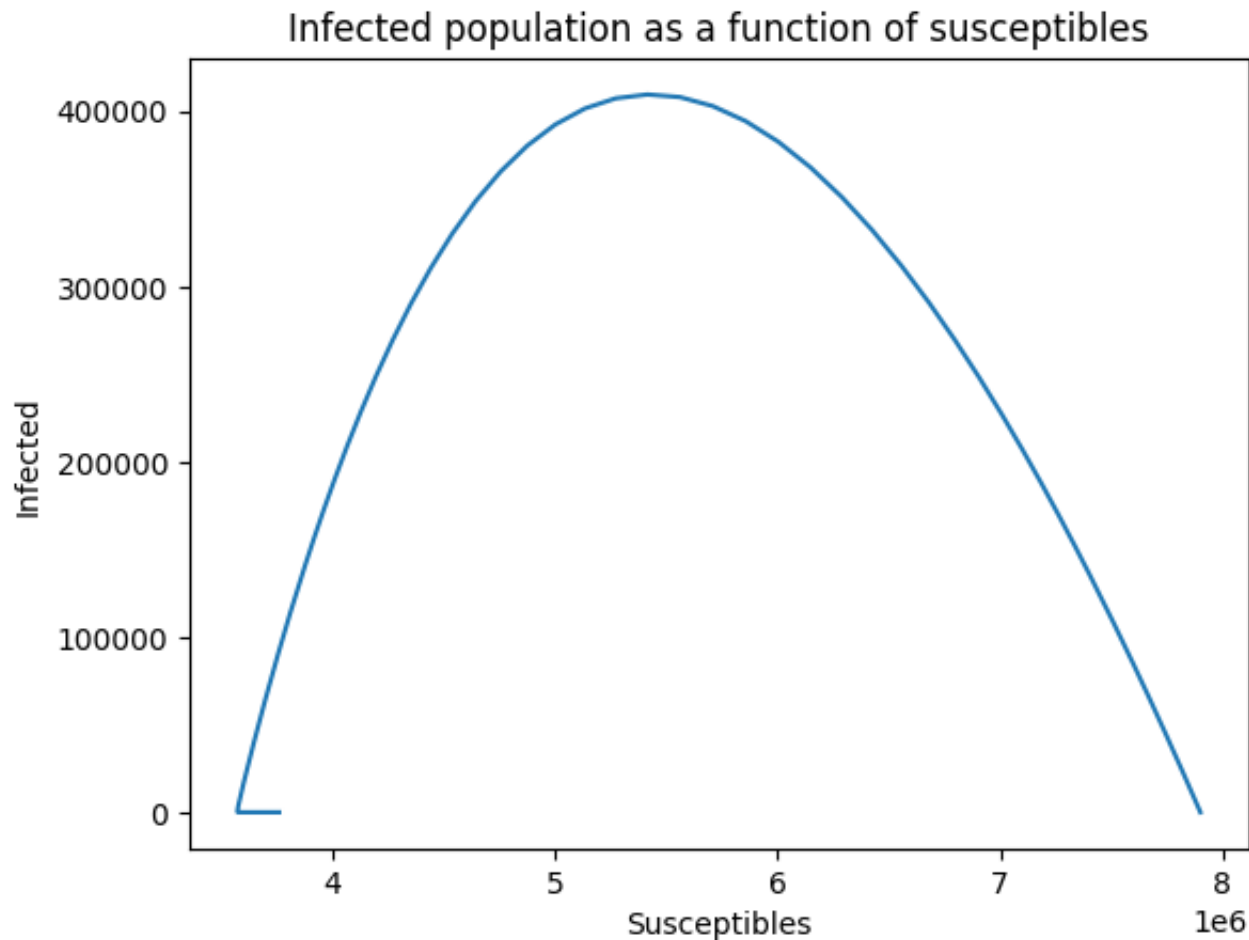
```

```
plt.xlabel('nb of days')
plt.ylabel('population')
plt.legend(loc="center left")
plt.show()

# Plot infected as a function of susceptibles.
plt.plot(Sval,Ival)
plt.title('Infected population as a function of susceptibles')
plt.xlabel('Susceptibles')
plt.ylabel('Infected')
plt.show()
```

Part 2: Population groups over 4 years with births and deaths





```
In [10]: # Define the function for the SIR model that doesn't assume a constant population, so incorporates
def variableimmSIR(alpha, beta, mu, w, initialval, Tfinal, gamma):
    Sval = np.zeros(Tfinal + 1) # creates a zero vector that holds S values for each day
    Ival = np.zeros(Tfinal + 1) # creates a vector of zeros for the I values for each day
    Rval = np.zeros(Tfinal + 1) # zero vector for the R values; populates for each day
    Mval = np.zeros(Tfinal + 1) # zero vector for the total population; populates for each day

    Sval[0] = initialval[0] # sets the first value of S to the initial value of S
    Ival[0] = initialval[1] # sets the first value of I to the initial value of I
    Rval[0] = initialval[2] # sets the first value of R to the initial value of R
    Mval[0] = Sval[0] + Ival[0] + Rval[0] # sets the first value of M to the total population at th
```

```

#loop over the time steps
for i in range(Tfinal): # creates a loop that goes through each day from 1 to Tfinal
    Mn = Sval[i] + Ival[i] + Rval[i] # sets M to the total population for each day
    Sval[i+1]=Sval[i]-((alpha/Mn)*Sval[i]*Ival[i]) + (mu*Mn) - (mu*Sval[i]) + (w*Rval[i]) # add
    Rval[i+1]=Rval[i]+(beta*Ival[i]) - (mu*Rval[i]) - (w*Rval[i]) # subtracts the death rate fr
    Ival[i+1] = Ival[i]+((alpha/Mn)*Sval[i]*Ival[i]) - (beta*Ival[i]) - ((mu + gamma)*Ival[i])

    Mval[i + 1] = Sval[i] + Ival[i] + Rval[i] # updates the total population for each day by ad
return Sval, Rval, Ival, Mval # returns the vectors of S, R, and I values for each day

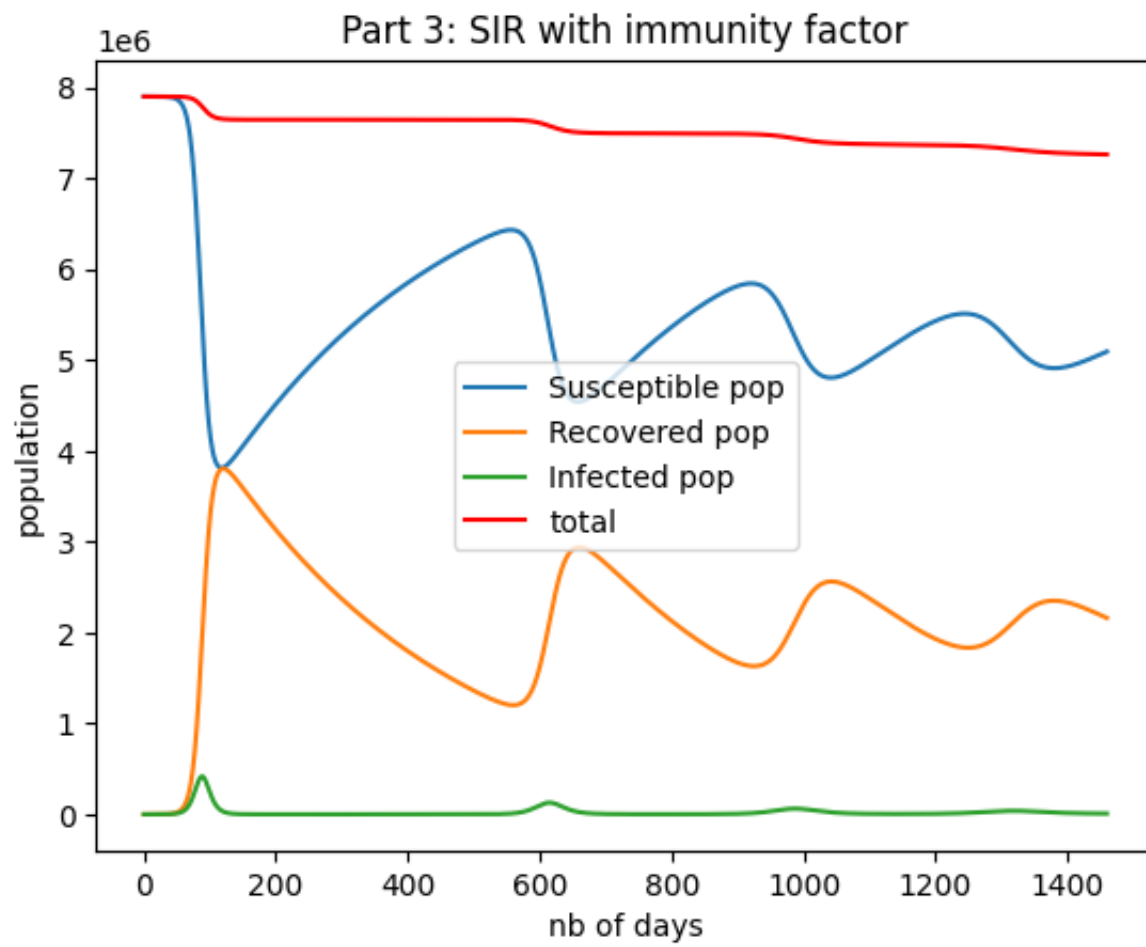
# Set variables to use in the equations
w = 1/365 # immunity rate
alpha = 1/2 # how quick recovery is
beta = 1/3 # how quick the disease spreads
Tfinal = 4 * 365 # number of days
Tval = np.arange(0,Tfinal + 1) # Uses matlab code to make a vector from time 0 to Tfinal with a ste
initialval = np.array ([7.9*10**6-10,10,0]) # sets the S, I, and R initial values in a vector

# Run the variableimmSIR function to get the S, I, and R values for each day
[Sval,Rval,Ival, Mval] = variableimmSIR(alpha,beta, mu, w, initialval,Tfinal, gamma) # calls create

# Plot the S, I, and R values against population vs days
plt.plot(Tval,Sval,label="Susceptible pop")
plt.plot(Tval,Rval,label="Recovered pop")
plt.plot(Tval,Ival,label="Infected pop")
plt.plot(Tval, Mval, label="total", color='red')
plt.title('Part 3: SIR with immunity factor')
plt.xlabel('nb of days')
plt.ylabel('population')
plt.legend(loc="best")
plt.show()

# Plot infected as a function of susceptibles.
plt.plot(Sval,Ival)
plt.title('Infected population as a function of susceptibles')
plt.xlabel('Susceptibles')
plt.ylabel('Infected')
plt.show()

```



Infected population as a function of susceptibles

